Metodologias Ágeis - SCRUM

UNIFEI | Universidade Federal de Itajubá
Instituto de Ciências Tecnológicas
ECO – <Análise e Desenvolvimento de Software II>
eduardo.felipe@unifei.edu.br
fabiana.costaguedes@unifei.edu.br
rossana.junqueira@unifei.edu.br
wendelldiniz@unifei.edu.br

Versão: 25/06/2025

Histórico

- Nasceram em meados de 1990;
- Motivados por uma reação aos “métodos pesados”;
- Em 2001, em uma reunião com 17 profissionais que já trabalhavam com os chamados métodos leves, surgiu e propagou-se os paradigmas de desenvolvimento de softwares ágeis;
- Durante a reunião, eles perceberam que apesar de cada um utilizar uma prática diferente ou adaptada à sua necessidade, existia um conjunto de princípios básicos que parecia ter sito respeitado por todos quando os projetos eram bem-sucedidos;
- Decidiram redigir um documento único para descrever o “Manifesto ágil”.

Manifesto Ágil

- Criado considerando a velocidade demandada para novos sistemas de software;
- Com objetivo de abandonar métodos antigos que se mostravam ultrapassados:
  - Uso de hardwares mais avançados, linguagens de programação, ambientes de desenvolvimento e necessidades organizacionais.
- Documento que encoraja o uso de melhores métodos de desenvolver sistemas de software, contento um conjunto de princípios que definem critérios para os processos de desenvolvimento ágil de sistemas.

Princípios

1. **A prioridade é satisfazer ao cliente por meio de entregas contínuas e frequentes de software de valor.**
   - Esta visão evidencia que o cliente é o principal foco das equipes de desenvolvimento ágil, devendo entregá-lo de forma rápida e eficiente.
2. **Mudanças de requisitos são bem-vindas, mesmo em uma fase avançada de projeto. Processos ágeis esperam que a mudança traga uma vantagem competitiva ao cliente.**
   - Processos ágeis não se incomodam caso projetos mudem ao longo do seu desenvolvimento, contudo as mudanças devem ser aceitas somente com o intuito de agregar valor e vantagens competitivas ao negócio do cliente. Mudanças que não colaboram com a evolução do projeto devem ser descartadas.

Princípios

3. **Entregas com frequência de software funcional, sempre na menor escala de tempo, de algumas semanas a alguns meses, preferindo sempre um período curto.**
   - Tempo de desenvolvimento é algo que deve se flexibilizar considerando as necessidades do cliente e da equipe de desenvolvimento. Sempre se deve buscar entregar o software funcional, com qualidade, o mais breve possível.
4. **As equipes de negócio e de desenvolvimento devem trabalhar juntas diariamente durante o projeto.**
   - Deve existir uma comunicação frequente entre a equipe de desenvolvimento e os principais interessados no projeto. O cliente deve sempre estar presente durante o acompanhamento do projeto.

Princípios

5. **Construa projetos objetivando manter uma equipe motivada, fornecendo ambiente, apoio e confiança necessários para realizar o trabalho.**
   - A equipe de desenvolvimento, os líderes e gerentes de projetos devem transmitir confiança um ao outro. Equipes ágeis devem ter características autogerenciáveis em um ambiente de trabalho organizado e estimulante, prezando pelo crescimento de seus integrantes e pela manutenção das qualidade do trabalho.

Princípios

6. **A maneira mais eficiente de a informação circular entre a equipe de desenvolvimento é por uma conversa cara a cara.**
   - O cliente sempre deve estar presente ao se discutirem aspectos relacionados ao sistema que está sendo desenvolvido. Metodologias ágeis estimulam a realização de reuniões mais frequentes e rápidas, que tornam mais fácil o entendimento das necessidades do cliente e do negócio, evitando inadequações.
7. **Ter um software funcionando é a medida primária de progresso.**
   - Entregar um software que não funcione é o mesmo que não entregá-lo. Espera-se que na medida em que as funcionalidades sejam validadas, sejam implementadas e entregues ao cliente.

Princípios

8. **Processos ágeis promovem o desenvolvimento sustentável. Patrocinadores, desenvolvedores e usuários devem ser capazes de manter um ritmo constante.**
   - O ritmo de entrega de novas funcionalidades consegue ser mantido, incluindo melhorias contínuas, se um ambiente organizado for proporcionado, estimulando a comunicação com foco nas prioridades do projeto.
9. **Atenção contínua à excelência técnica e a um bom projeto aumenta a agilidade.**
   - Seguir padrões de projeto que prezam pela qualidade e design é importante para manter a excelência técnica da equipe de desenvolvimento. Seguindo esses princípios, é possível entregar software com qualidade rapidamente.

Princípios

10. **Simplicidade é essencial.**

- Estimula-se a busca por soluções simples para os requisitos do cliente, que possam eventualmente se aprimoradas posteriormente. Arquiteturas complexas podem não agregar valos ao cliente e contribuir com atraso nas entregas.

11. **As melhores arquiteturas, requisitos e projetos provêm de equipes organizadas.**

- As metodologias ágeis estimulam a formação de equipes auto-organizáveis, pois conseguem se adaptar mais facilmente às mudanças de direção do negócio proposto pelos patrocinadores do projeto. Equipes auto-organizáveis reinventam e reestruturam o negócio com criatividade e conforme a demanda dos clientes.

Princípios

12. **Em intervalos regulares, a equipe deve refletir sobre como se tornar mais eficaz, então sintoniza e ajusta seu comportamento.**
    - Reflexões periódicas sobre o desempenho da equipe são importantes para identificar processos falhos e desnecessários, bem como traçar planos de melhoria futura, que devem ser propostos pela própria equipe.

- Com base nos princípios elencados, cada um dos métodos ágeis existentes oferece um conjunto de atividades que podem ser adotadas durante um processo de desenvolvimento de software.
- A ideia principal é redefinir continuamente as prioridades em um projeto de software.

Principais Motivações

- O uso de metodologias, independente de quais sejam, é um consenso, pois impõe o usos de processos disciplinados durante o desenvolvimento de um software.
- O uso de processos disciplinados têm o objetivo de tornar o desenvolvimento mais previsível e eficiente.
- As metodologias ágeis surgiram em resposta a metodologias tradicionais mais “engessadas”, proporcionando mudanças de ênfase significativas e estimulando pequenos grupos de desenvolvedores a adotar um processo disciplinado para projetos de qualquer natureza.

Principais Motivações

- Metodologias ágeis:
  - São menos centradas em documentações, sendo mais voltadas ao código-fonte do sistema de software.
  - São adaptativas em vez de predeterminantes.
  - Aceitam mudanças ao longo do desenvolvimento de maneira natural, adaptando seus processos às mudanças exigidas.
  - São orientados a pessoas e não a processos. Nenhum processo pode ser equivalente à habilidade dos integrantes da equipe de desenvolvimento. Os processos existentes nos métodos ágeis têm o papel de dar suporte à equipe de desenvolvimento.

Metodologia SCRUM

- Fortemente influenciada por boas práticas adotadas pela indústria japonesa, em especial por princípios da manufatura enxuta implementados pelas companhias Honda e Toyota. Para elas, projetos que usam equipes pequenas e multidisciplinares produzem melhores resultados.
- A denominação surgiu da associação dessas equipes de projeto altamente eficazes com um típica formação do evento esportivo rugby denominada scrum.
- No rugby, essa formação é utilizada após determinado incidente ou quando a bola sai de campo, ou seja, é utilizada para reiniciar o jogo, reunindo todos os jogadores.
- No rugby cada time age em conjunto, cada membro desempenha um papel específico e todos se ajudam em busca de um benefício comum.

Características do SCRUM

- Segue os princípios do manifesto ágil.
- Baseia-se em seis características:
  - Flexibilidade dos resultados;
  - Flexibilidade dos prazos;
  - Times pequenos;
  - Revisões frequentes;
  - Colaborações; e
  - Orientação a objetos.

Características do SCRUM

- Não existe uma solução mágica para problemas complexos, mas existe um consenso de que podemos usar o SCRUM para:
  - Desenvolvimento complexos em que os requisitos mudam rapidamente e constantemente;
  - Gerenciar e controlar o desenvolvimento do trabalho;
  - Tornar a equipe autogerenciável e funcional;
  - Implementar o conceito iterativo e incremental no desenvolvimento de software e/ou produtos;
  - Identificar causas de problemas e remover impedimentos;
  - Valorizar os indivíduos.

Características do SCRUM

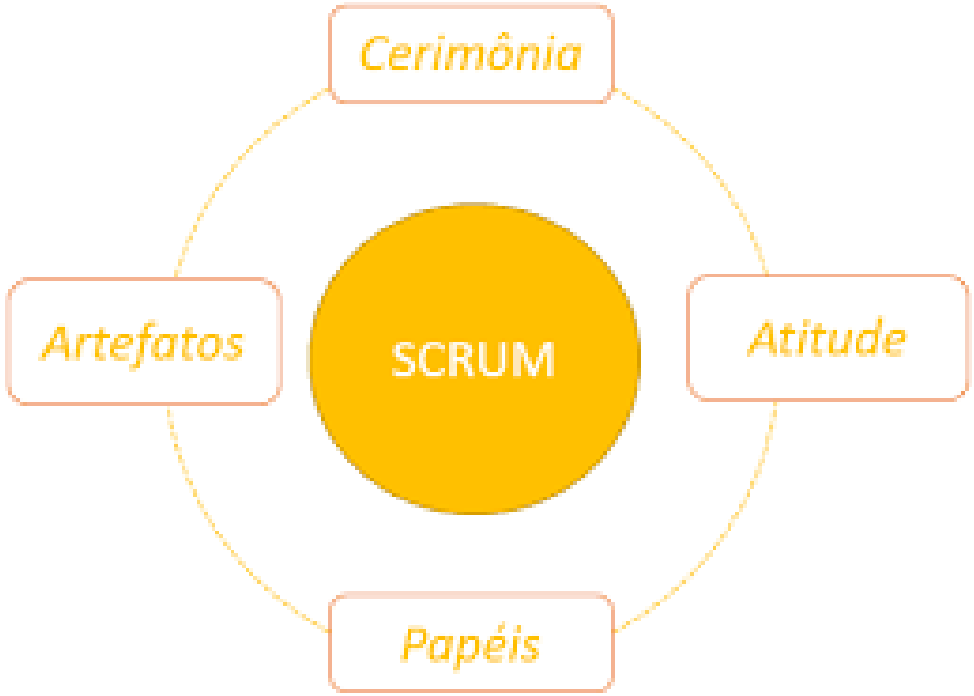
- Não existe uma solução mágica para problemas complexos, mas existe um consenso de que podemos usar o SCRUM para:
  - Desenvolvimento complexos em que os requisitos mudam rapidamente e constantemente;
  - Gerenciar e controlar o desenvolvimento do trabalho;
  - Tornar a equipe autogerenciável e funcional;
  - Implementar o conceito iterativo e incremental no desenvolvimento de software e/ou produtos;
  - Identificar causas de problemas e remover impedimentos;
  - Valorizar os indivíduos.

Características do SCRUM

- Sua característica iterativa e incremental permite utilizá-lo no desenvolvimento de qualquer produto ou no gerenciamento de qualquer trabalho.
- O desenvolvimento iterativo e incremental é uma estratégia de planejamento em que o produto é construído em partes (iterações). Ao término de cada parte, é feito um novo incremento até que o desenvolvimento seja completado.

Características do SCRUM

- Também é conhecido pelo fato de estruturar seu funcionamento por ciclos, chamados de *sprints*, que representam iterações de trabalho com duração variável, geralmente de duas a quatro semanas.

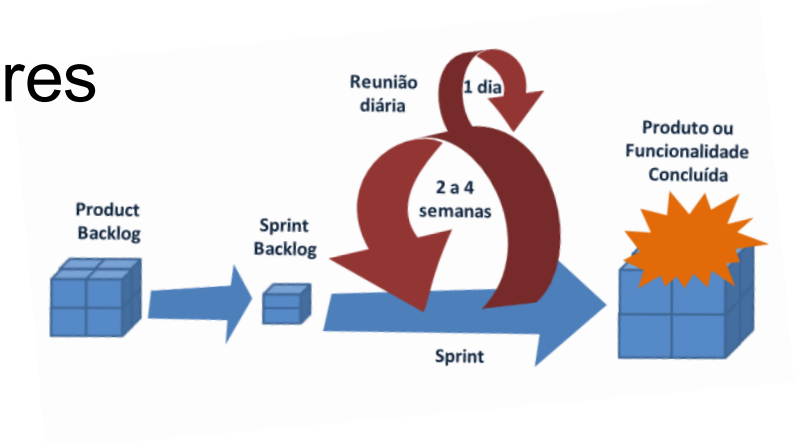


Características do SCRUM

- Também estabelece um conjunto de práticas e regras que devem ser cumpridas pela equipe, em seus respectivos **papéis**.
  - Muitos acreditam ser prudente incluir o cliente entre os papéis, uma vez que de fato ele tem uma participação importante no processo de desenvolvimento.
- Podemos identificar quatro **cerimônias** existentes, que são conduzidas periodicamente ao longo do desenvolvimento.
- Como resultado dessas **cerimônias** são gerados três **artefatos** básicos.
- Tem como princípio básico o reconhecimento de que os clientes podem mudar de ideia ao longo do desenvolvimento do projeto, simplesmente porque eles querem ou precisam.

Ciclo de Desenvolvimento do SCRUM

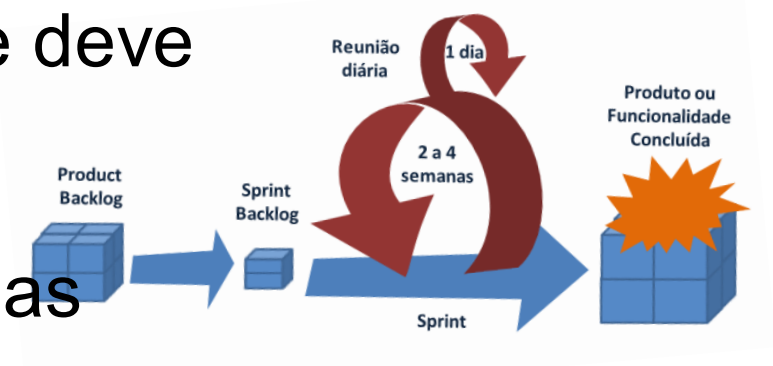
- No início de cada projeto, clientes e desenvolvedores se reúnem com o objetivo de definir um resumo (*Backlog*) do produto (que representa a lista de requisitos).



- Neste momento também são estimados os custos do projeto e é feita uma análise dos riscos, bem como as ferramentas de trabalho e os integrantes da equipe são escolhidos e são definidas as datas de entregas dos resultados a partir de priorizações sinalizadas pelo cliente.
- Esse período também é utilizado para escolher o *SCRUM Master*.

Ciclo de Desenvolvimento do SCRUM

- Depois de definido o *Product Back*log, a equipe deve se dedicar à definição da *Sprint Backlog*, que contém uma lista de atividades que serão realizadas na próxima sprint, momento em que também são definidas as responsabilidades de cada membro do time.



- Após os desenvolvedores discutirem quais padrões serão adotados, as atividades de análise, codificação e testes devem se iniciar.
- Ao final de cada *sprint* uma funcionalidade do produto deve ser apresentado ao cliente.

Sprints

- Correspondem a iterações.
- O objetivo é gerar um produto “entregável” de valor para o cliente, que foi previamente combinado com ele.
- Cada *sprint* deve ocorrer em um período de 2 a 4 semanas.
- O produto é projetado, codificado e testado durante a *sprint*.
- Durante a *sprint*, nenhuma mudança deve ocorrer.
- Cada *sprint* é precedida por uma reunião de planejamento, com o objetivo de identificar quais tarefas serão executadas durante a *sprint* e assumir um objetivo para a *sprint*, resultando em um incremento de produto potencialmente entregável para o cliente.

Sprints

- As tarefas escolhidas para fazerem parte de uma *sprint* devem ser retiradas do *Product Backlog*.
- No final de cada *sprint*, outra reunião deve ser realizada, objetivando revisar o que foi feito, avaliar o progresso e identificar lições aprendidas para serem usada na próxima *sprint*.
- Cada *sprint* é uma iteração que segue um ciclo conhecido como PDCA (*Plan*, *Do*, *Check* e *Act*), podendo ser utilizado em qualquer área de atuação.

Papéis – *Product Owner*

- O “Dono” do produto representa o cliente e é responsável por garantir que a equipe SCRUM agregue valor ao negócio.
- Desempenha o papel de moderador entre os interesses do cliente e do *Team*.
- É responsável por:
  - Definir a visão e as funcionalidades do produto;
  - Definir as prioridades;
  - Elaborar e mante o *Product Backlog*;
  - Definir as prioridades e o ROI (*Return of Investment*);
  - Decidir sobre as datas de lançamento do produto;
  - Representar o cliente;
  - Aceitar ou rejeitar os resultados dos trabalhos.

Papéis – *SCRUM Master*

- É o representante do cliente no projeto e também desempenha um papel importante de facilitador, responsável pela remoção de impedimentos que eventualmente surjam durante o desenrolar do desenvolvimento.
- Atua na definição de funcionalidades de acordo com seu valor para o cliente, planejando e elaborando em conjunto com o *Product Owner* uma lista de prioridades.
- Desempenha um papel de responsabilidade técnica na condução do projeto, devendo proporcionar mecanismos preferencialmente informatizados de comunicação entre seus integrantes.
- Deve proteger a equipe e mantê-la focada em suas tarefas.

Papéis – *SCRUM Master*

- É responsável por:
  - Desempenhar o papel de líder, representando a gerência do projeto;
  - Remover impedimentos;
  - Ajudar o dono do projeto (*Product Owner*) com o resumo do produto (*Product Backlog*);
  - Ser o facilitador da equipe SCRUM, garantindo sua plena produtividade;
  - Garantir a colaboração entre os diversos papéis e funções;
  - Atuar como escudo para interferências externas;
  - Aplicar os valores e as práticas SCRUM;

Papéis – *TEAM ou equipe SCRUM*

- É o time (5 a 9 pessoas) responsável pelo desenvolvimento do projeto, que deve possuir uma característica multifuncional.
- O SCRUM estimula a comunicação verbal entre todos os membros do projeto.
- Os integrantes do time só devem ser trocados após o término de uma *sprint*.
- É responsável por:
  - Fazer as estimativas necessárias;
  - Definir as tarefas que serão realizadas;
  - Desenvolver o produto;
  - Garantir a qualidade do produto;
  - Apresentar o produto ao cliente.
- Os integrantes do time podem possuir experiências diversas.

Papéis – Clientes

- O cliente deve participar de tarefas relacionadas à implementação da lista de funcionalidades e atuar ao longo do desenvolvimento do projeto por meio de análises periódicas dos subprodutos gerados.

Cerimônias

- São reuniões que ocorrem em momentos diferentes de uma determinada *sprint*.
- São elas:
  - Planejamento da *sprint* (*Sprint Planning Meeting*)
  - Reunião diária (*Daily Meeting* ou *Daily SCRUM*)
  - Revisão da *sprint* (*Sprint Review*)
  - Retrospectiva da *sprint* (*Sprint Retrospective*)

Cerimônias – Planejamento da *Sprint*

- É a primeira reunião do projeto e todos precisam participar.
- Deve durar no máximo 8 horas.
- É a reunião em que o *Product Owner* planeja e elabora a lista de prioridades que devem ser cumpridas pelo projeto, dividindo a reunião em duas partes:
  - Parte 1: O *Product Owner* define suas prioridades, seleciona os itens do *backlog* e a meta da *sprint*.
  - Parte 2: A equipe define a *sprint backlog*, que é um documento que contêm as tarefas necessárias para cumprir a meta.

Cerimônias – Reunião diária

- Reunião simples e importante – grande diferencial do SCRUM.
- Cada membro do time deve responder sobre o que já fez, sobre o que pretende fazer e se há algum impedimento para a conclusão da(s) tarefa(s) sob sua responsabilidade.
- Deve ser rápida – máximo de 15 minutos.
- Participantes: Time e *SCRUM Master*.

Cerimônias – Revisão da *sprint*

- Reunião de balanço sobre tudo o que foi feito durante uma *sprint*.
- O time deve mostrar os resultados da *sprint* para o *Product Owner* e seus convidados.
- Devem ser apresentados somente itens que estiverem 100% prontos.
- Dinâmica da reunião:
  - Um membro do time apresenta a meta do *sprint*, os itens do *Product Backlog* com os quais o time se comprometeu e todos os itens que foram concluídos.
  - Em seguida, os membros do time fazem uma demonstração funcional dos itens desenvolvidos.
- Duração estimada de 4 horas.
- Devem participar: *Product Owner*, *SCRUM Master*, Equipe *SCRUM* e outros convidados (se necessário).
- Deve acontecer sempre no final da *sprint*.
- Após a apresentação, o *Product Owner* tem o direito de aceitar ou rejeitar a sprint.
- Qualquer necessidade de mudança ou inserção de novas funcionalidades será incorporada ao *Product Backlog* em um momento oportuno.

Cerimônias – Retrospectiva da *sprint*

- Seu objetivo é verificar o que houve de bom e o que poder ser melhorado em uma *sprint*.
- São avaliados aspectos relacionados ao trabalho em equipe, pontos positivos e negativos, e feitas reflexões sobre estratégias de melhoria que podem ser adotadas.
- Devem participar: O time e o *SCRUM Master*. Se convidado, o *Product Owner*.
- Todos os membros do time devem responder: O que foi bom durante a *sprint* e o que pode ser melhorado na próxima?
- O SCRUM Master deve tomar nota de tudo e o time deve priorizar os itens apontados em uma ordem ideal de mudança.
- A retrospectiva garante a melhoria contínua do processo.
- Deve acontecer logo após a revisão da sprint, com duração aproximada de 3 horas.

Artefatos

- São produzidos em momentos diferentes de uma determinada sprint.
- São eles:
  - *Product Backlog*
  - *Sprint Backlog*
  - *Burndown Chart*

Artefatos – *Product Backlog*

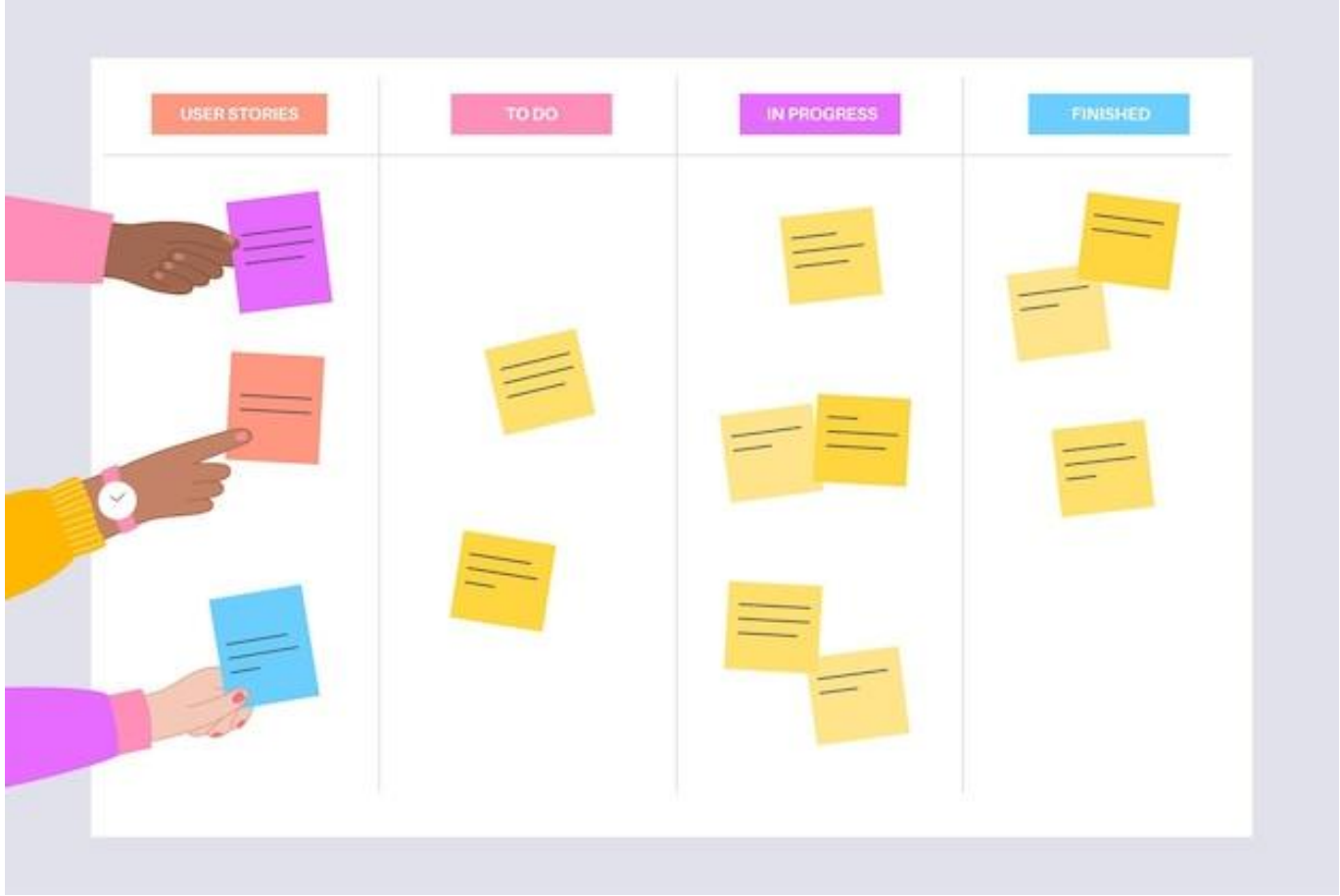
- Documento que representa a visão do produto, contendo todos os itens que devem ser desenvolvidos durante o projeto.
- Lista de prioridades feitas logo no início do projeto, com o objetivo de esclarecer e elencar o que deve ser entregue para o cliente.
- Deve ser criado e mantido pelo *Product Owner*, que tem a liberdade de alterar esse documento quando quiser, desde que os itens alterados não estejam na *sprint* que estiver sendo desenvolvida no momento.
- Seria o equivalente ao nosso levantamento de requisitos, visto anteriormente.

Artefatos – *Sprint Backlog*

- Oriundo da *Sprint Planning Meeting* e representa todas as tarefas que devem ser desenvolvidas durante uma *sprint* ou iteração.
- Não deve sofrer alterações.
- Cada item deve ser detalhado em tarefas e cada uma dessas tarefas deve ter uma estimativa de esforço, neste caso, em horas.
- Para melhor visibilidade, pode ser apresentado na forma de quadro.
- É interessante que o time só comece a trabalhar o segundo item sempre após a conclusão do primeiro.

Artefatos – *Sprint Backlog*

- Quadro de tarefas – *Task Board*
  - Utilizado para o acompanhamento das *sprints*.



Artefatos – Gráfico *Burndown*

- Tem como objetivo mostrar o esforço restante para a conclusão da iteração, bem como mostrar o quão próximo ou distante está de atingir a meta.
- Quando o gráfico está acima da linha planejada, o time está distante de atingir a meta.
- Em cima da linha planejada, o time está em um fluxo de trabalho ideal.
- Abaixo da linha planejada, o time está superando as expectativas.
- O gráfico deve ser atualizado diariamente.

Referências

- SBROCCO, José Henrique Teixeira de Carvalho; MACEDO, Paulo Cesar. Metodologias Ágeis: Engenharia de Software sob medida. São Paulo: Érica, 2012.